

CHAPTER V

SYSTEM DEVELOPMENT

System Development is a continuation phase of the previous phase, System Designing. Here the previously designed system will be implemented as form of codes. The system will also be tested for bugs and other problems after development phase. The source code of DietProject will be described in detail here.

5.1.Database Filling

DietProject relies heavily on its stored database. For the database purpose, DietProject will having their own database. Saved in the User phone. It is also noted that in the future Diet Project should rely on external source of database and only use user's memory when needed. The data source of Diet Project originated from [HTTP://www.invive.com/calorie.html](http://www.invive.com/calorie.html) and will be updated when sites update their data. Due to this external database source, the accuracy and the content depends heavily on those websites. In further development later, Diet Project should have an independent database from the community.

5.2.Database Creation

5.2.1. Database Models

Before creating databases for Diet project applications first we must make the data models filled with Setter and Getter functions. There are 6 models classes CalDiaryModels, BreakfastModels, FoodModels, and UserDataModels. The main use of models class is to constructs

the data to be stored and take from the database so it's easier for the programmer to manipulate the data content.

5.2.2. Database Operations Class

All the database interactions will be handle by a Helper class so it will be easier for us to do all database related operations. The first part of this class is creating the variables names and creating the database in the phone

```
// Database Name
private static final String DATABASE_NAME = "DietProject2";

// Table Names
private static final String TABLE_FOOD = "food";
private static final String TABLE_WORK_OUT = "work_out";
private static final String TABLE_USER_DATA = "user_data";
private static final String TABLE_TARGET_PROGGRES = "target_proggres";
private static final String TABLE_FOOD_DIARY = "food_diary";
private static final String TABLE_WORKOUT_DIARY = "workout_diary";
```

Figure 5.1 Variable declarations

After done with creating the variables then creating the tables.

```
// Table Create Statements
private static final String CREATE_TABLE_FOOD =
    "CREATE TABLE " + TABLE_FOOD
    + "(" + KEY_ID + " INTEGER,"
    + KEY_FOOD_NAME + " TEXT PRIMARY KEY,"
    + KEY_FOOD_CALORIE + " INTEGER,"
    + KEY_FOOD_PORTION + " INTEGER,"
    + KEY_FOOD_PORTION_NAME + " TEXT"+ ")";

@Override
public void onCreate(SQLiteDatabase db) {

    // creating required tables
    db.execSQL(CREATE_TABLE_FOOD);
    db.execSQL(CREATE_TABLE_WORKOUT);
    db.execSQL(CREATE_TABLE_USER_DATA);
    db.execSQL(CREATE_TABLE_FOOD_DIARY);
    db.execSQL(CREATE_TABLE_WORKOUT_DIARY);
    db.execSQL(CREATE_TABLE_TARGET_PROGGRES);
```

Figure 5.2 Table create code

After declaring the variables and creating the tables the database also need to be programmed for updating databases. DietProject databases are mainly to drop the older table and recreate the new tables from scratch.

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    // on upgrade drop older tables
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_FOOD);
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_WORK_OUT);
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_USER_DATA);
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_FOOD_DIARY);
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_WORKOUT_DIARY);
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_TARGET_PROGGRES);
```

Figure 5.3 Update database code

There are 5 more create table statements according to the ERD there are food, workout, User data, Food diary, Workout diary, and progress. After that we can process the database using CRUD (Create, Read, Update, and Delete) operations.

```

    Creating a food
    */
    public long createFood (FoodModels food ) {

        SQLiteDatabase db = this.getWritableDatabase();

        ContentValues values = new ContentValues();
        values.put(KEY_FOOD_NAME, food.getName());
        values.put(KEY_FOOD_CALORIE, food.getCalorie());
        values.put(KEY_FOOD_PORTION, food.getPortion());
        values.put(KEY_FOOD_PORTION_NAME, food.getPortsize());

        // insert row
        long food_id = db.insert(TABLE_FOOD, null, values);
        return food_id;
    }

```

Figure 5.4 Create data operations

Createfood in figure 5.4 function only accept the data using FoodModels constructor. Using this it can be sure that the inputted data is not a null value. By first assign the data to a models than it can be sure for the input is not a null value.

To Read a single data from the table we can get a single food this function accept the name of the food to be load and return the data of food models. We can see it in Figure 5.5.

```

public FoodModels getFood (String foodname) {
    SQLiteDatabase db = this.getReadableDatabase();
    String selectQuery = "SELECT * FROM " + TABLE_FOOD + " WHERE "+
    KEY_FOOD_NAME + " = '" + foodname + "'";

    Cursor c = db.rawQuery(selectQuery, null);

    if (c != null)
        c.moveToFirst();

    FoodModels fm = new FoodModels();
    fm.setId(c.getInt(c.getColumnIndex(KEY_ID)));
    fm.setName((c.getString(c.getColumnIndex(KEY_FOOD_NAME))));
    fm.setCalorie((c.getInt(c.getColumnIndex(KEY_FOOD_CALORIE))));
    fm.setPortion(c.getInt(c.getColumnIndex(KEY_FOOD_PORTION)));
    fm.setPortsize(c.getString(c.getColumnIndex(KEY_FOOD_PORTION_NAME)));
    return fm;
}

```

Figure 5.5 Read data operations

To get more than a data or a list of all food we assign the result of this function to an array list of FoodModels as can be seen in Figure 5.6.

The benefit using this approach is that we can get a full food models and can easily load any of the list members to be processed further. The return value is an array list of FoodModels this code mainly used in for the list in the activity page (Breakfast, Lunch, Dinner, Snack, and Workout.)

DietProject implement two list the other one are shown in Figure 5.7

```

/**
 * getting all Food and return in an arraylist
 */
public ArrayList<FoodModels> getAllFood () {
    ArrayList<FoodModels> foodModelses = new ArrayList<FoodModels>();
    String selectfoodQuery = "SELECT * FROM " + TABLE_FOOD;

    Log.e(LOG, selectfoodQuery);

    SQLiteDatabase db = this.getReadableDatabase();
    Cursor c = db.rawQuery(selectfoodQuery, null);

    // looping through all rows and adding to list
    if (c.moveToFirst()) {
        do {
            FoodModels fm = new FoodModels();
            fm.setId(c.getInt(c.getColumnIndex(KEY_ID)));
            fm.setName((c.getString(c.getColumnIndex(KEY_FOOD_NAME))));
            fm.setCalorie((c.getInt(c.getColumnIndex(KEY_FOOD_CALORIE))));
            fm.setPortion(c.getInt(c.getColumnIndex(KEY_FOOD_PORTION)));
            fm.setPortsize(c.getString(c.getColumnIndex(KEY_FOOD_PORTION_NAME)));

            // adding to list
            foodModelses.add(fm);
        } while (c.moveToNext());
    }

    return foodModelses;
}

```

Figure 5.6 getAllFood Functions

Using this approach we only load the name and calorie amount of the food instead the whole Food models. The benefit of this approach is I will resulted in a lighter and faster loading time and improve Users Experience.

```

public ArrayList<Map<String, String>> getAllFoodSpecial () {
    String fnm;
    int fcl;
    ArrayList<Map<String, String>> foodModelses = new ArrayList<Map<String,
String>>();

    String selectfoodQuery = "SELECT * FROM " + TABLE_FOOD + " ORDER BY " +
KEY_FOOD_NAME + " COLLATE NOCASE ";

    Log.e(LOG, selectfoodQuery);

    SQLiteDatabase db = this.getReadableDatabase();
    Cursor c = db.rawQuery(selectfoodQuery, null);

    // looping through all rows and adding to list
    if (c.moveToFirst()) {
        do {
            FoodModels fm = new FoodModels();
            fm.setId(c.getInt(c.getColumnIndex(KEY_ID)));
            fm.setName((c.getString(c.getColumnIndex(KEY_FOOD_NAME))));
            fm.setCalorie((c.getInt(c.getColumnIndex(KEY_FOOD_CALORIE))));
            fm.setPortion(c.getInt(c.getColumnIndex(KEY_FOOD_PORTION)));
            fm.setPortsize(c.getString(c.getColumnIndex(KEY_FOOD_PORTION_NAME)));

            fnm = c.getString(c.getColumnIndex(KEY_FOOD_NAME));
            fcl = c.getInt(c.getColumnIndex(KEY_FOOD_CALORIE));
            // adding to list
            foodModelses.add(putData(fnm,fcl));
        } while (c.moveToNext());
    }

    return foodModelses ;
}

private HashMap<String, String> putData(String name, Integer calorie) {
    HashMap<String, String> item = new HashMap<String, String>();
    item.put("name", name);
    item.put("calorie", String.valueOf(calorie));
    return item;
}

```

Figure 5. 7 getAllFood Special

Beside creating and reading Dietapplication also able to uupdate food as it's deem necessary.

```

/**
 * Updating a food
 */
public int updateFood (FoodModels foodModels) {
    SQLiteDatabase db = this.getWritableDatabase();

    ContentValues values = new ContentValues();
    values.put(KEY_FOOD_NAME, foodModels.getName());
    values.put(KEY_FOOD_CALORIE, foodModels.getCalorie());
    values.put(KEY_FOOD_PORTION, foodModels.getPortion());
    values.put(KEY_FOOD_PORTION_NAME, foodModels.getPortsize());

    // updating row
    return db.update(TABLE_FOOD, values, KEY_ID + " = ?", new String[] {
String.valueOf(foodModels.getId()) });
}

```

Figure 5. 8 Update item Code.

Update function can be seen at Figure 5.8 and Delete function can be seen at Figure

5.9 as the name suggest their job is to delete and Update the data

```

/*
 * Deleting a food
 */
public void deleteFood (long food_id ) {
    SQLiteDatabase db = this.getWritableDatabase();
    db.delete(TABLE_FOOD, KEY_ID + " = ?", new String[] { String.valueOf(food_id) });
}

```

Figure 5. 9 Delete item Code

5.3.Application Activity

After we done with creating the databases for the application. The next is to create the application itself. It consists of several java classes.

5.3.1. Main Activity

We start with the main activity of the applications. The main activity will handling First time launches and show the user the Update user data in order to get User data first. It has two possible course of actions that is Update user data and Adding New Activity. Main Activity is have a special treatment for first time users.

```

if (prefs.getBoolean("firstrun", true)) {
    //do stuff here if first run
    prefsEditor.putString("datestart", currentDateTimeString).commit();
    prefsEditor.putInt("dayid", 1);
    Intent intent = new Intent(this, dbcreate.class);
    Intent intent = new Intent(this, UpdateUserData.class);
    startActivity(intent);
    //make sure to flag the boolean as false
    prefs.edit().putBoolean("firstrun", false).commit();
} else {
    if ((savedDate + (24 * 60 * 60 * 1000)) < System.currentTimeMillis()) {

```

Figure 5.10 First time run code

When the user are first time starting the application the application will check the SharedPreferences named firstrun and retrieve the Boolean value. Because the default value of a Boolean preferences then the value of firstrun definitely will be true. When user first time run the application then the value of datestart are set to current time and dayid set to 1. The value of datestart will then be compared each time the application run. If the difference is equal or more than 1 day than the value of dayid will be added by 1 as can be seen in Figure 5.6. After that Main activity will start 2 classes dbcreate.class and UpdateUserData.class. dbcreate as the name suggest this classes is responsible to create database that saved to the users phone. After creating the database than the UpdateUserData will ask the user to input the data to the database.

```

{
    dayid = Long.valueOf(prefs.getInt("dayid", 1));
    Toast.makeText(MainMenu.this, String.valueOf(dayid), Toast.LENGTH_SHORT).show();
    //if not first run, do something else
    DailyQuota = (TextView) findViewById(R.id.DailyQuota);
    CalBurned = (TextView) findViewById(R.id.CalBurned);
    CalAte = (TextView) findViewById(R.id.CalAte);
    CalRemaining = (TextView) findViewById(R.id.CalRemaining);

    DCQ = prefs.getString("Target", " 0 ");
    DailyCalQuota = prefs.getInt("DailyCalorieQuota", 0);
    DailyCalAte = db.getfooddaydata(dayid);
    DailyCalBurned = db.getworkoutdaydata(dayid);
    DailyCalRemaining = DailyCalQuota + DailyCalBurned - DailyCalAte;

    DailyQuota.setText(DCQ + " Calorie");
    CalBurned.setText(DailyCalBurned + " Calorie");
    CalAte.setText(DailyCalAte + " Calorie");
    CalRemaining.setText(DailyCalRemaining + " Calorie");
}

```

Figure 5.11 Main Activity code

In Main Activity the user will be showed to their daily calorie need and the amount of calorie target. The Daily calorie remaining is achieved by subtracting the sum of DailyCalQuota and DailyCalBurned with DailyCalAte the code can be seen at Figure 5.7.

```

prefsEditor.putLong("saveddate", savedDate);
dayid = Long.valueOf(prefs.getInt("dayid", 1));
DailyCalQuota = prefs.getInt("DailyCalorieQuota", 0);
DailyCalAte = db.getfooddaydata(dayid);
DailyCalBurned = db.getworkoutdaydata(dayid);
Integer i = dayid != null ? dayid.intValue() : null;
TargetProgress tp = new TargetProgress(i, DailyCalQuota, DailyCalAte, DailyCalBurned);
b = db.createTargetProgress(tp);
i = i + 1;
prefsEditor.putInt("dayid", i);
Toast.makeText(MainMenu.this, String.valueOf(i), Toast.LENGTH_SHORT).show();

```

Figure 5.12 Dayid check code

Main Activity also responsible to check the date to increment the dayid. Everytime Main activity is accessed then it will check the date and compare it with the latest entry of date. If the difference is more than or equal to 1 day than the dayid will be incremented.

5.3.2. Update user data

In Class Update user data user can input their personal data consists of Weight, Height, Age, Gender, Activity level, and User Target. After inputting these user can get their BMR (Basal Metabolismic rate), Daily Calorie Needs, and Daily Calorie Target.

```
switch(view.getId()) {
    case R.id.RadioButtonMan:
        if (checked)
            UserBmr = 10 * UserWeight + 6.25 * UserHeight - 5 * UserAge + 5 ;
        break;
    case R.id.RadioButtonWoman:
        if (checked)
            UserBmr = 10 * UserWeight + 6.25 * UserHeight - 5 * UserAge - 161 ;
        break;
}
```

Figure 5.13 BMR code

BMR counting used the formula from The Mifflin St Jeor Equation as can be seen at Figure 5.14

$$P = \left(\frac{10.0m}{1 \text{ kg}} + \frac{6.25h}{1 \text{ cm}} - \frac{5.0a}{1 \text{ year}} + s \right) \frac{\text{kcal}}{\text{day}}$$

Figure 5.14 Mifflin St Jeor Equation

After count the BMR we count the daily calorie needs by factoring the activity level.

```

boolean checked = ((RadioButton) view).isChecked();

switch(view.getId()) {
    case R.id.RadioLose1KG:
        if (checked)
            UserTarget = DailyCalorieNeeds - 1000 ;
        break;
    case R.id.RadioLoseHalfKG:
        if (checked)
            UserTarget = DailyCalorieNeeds - 500 ;
        break;
    case R.id.RadioKeepWeight:
        if (checked)
            UserTarget = DailyCalorieNeeds ;
        break;
    case R.id.RadioGainHalfKG:
        if (checked)
            UserTarget = DailyCalorieNeeds + 500 ;
        break;
    case R.id.RadioGain1KG:
        if (checked)
            UserTarget = DailyCalorieNeeds + 1000 ;
        break;
}

```

Figure 5. 15 Daily Calorie Need Code

Daily Calorie Need is the amount of how much the user are burn calories each day. Daily calorie need show how fast the calorie burn by normal means in the human bodies. Daily calorie need is the amount of calorie needed for the user to keep the weight. Figure 5.15 will show how the concept are implemented to the code. Lastly we add the factor from the user target vary from -1000 calorie until +1000 calorie. In theory adding 1000 calorie is equal to adding 1 kilo a week and Vice versa. So if the user want to gain 1KG of weight they need to add 1000 calorie in a week to their daily calorie needs. Figure 5.16 will show how the concept are implemented to the code

```

switch(view.getId()) {
    case R.id.RadioSedentary:
        if (checked)
            DailyCalorieNeeds = UserBmr * 1.2 ;
        break;
    case R.id.RadioLightlyActive:
        if (checked)
            DailyCalorieNeeds = UserBmr * 1.375 ;
        break;
    case R.id.RadioModeratelyActive:
        if (checked)
            DailyCalorieNeeds = UserBmr * 1.55 ;
        break;
    case R.id.RadioVeryActive:
        if (checked)
            DailyCalorieNeeds = UserBmr * 1.725 ;
        break;
    case R.id.RadioExtraActive:
        if (checked)
            DailyCalorieNeeds = UserBmr * 1.9 ;
        break;
}

```

Figure 5.16 User Target code

5.3.3. Add Activity

Here user can add new activity by choosing one of the available activity. There are 5 kind of activity that can be choose in add activity page.

5.3.4. Add Activity Items (Breakfast, Lunch, Dinner, Snack Workout)

Users can add their activity items Breakfast, Lunch, etc. There are 4 add activity classes mainly they are used to show the food / workout that have been done.

To show the user the food they already eaten Diet Project uses Listview as can be seen in Figure 5.17. This view will show the user the list of food they have ate. The list will load from table Food_Diary in the Database. In the activity page the listview are using the simple adapter with the scheme of simple_list_item_2.

Using this kind of approach the user can expect stability and more streamlined user experience.

```
final ListView listview = (ListView) findViewById(R.id.list_view);
ArrayList<Map<String, String>> list = db.getActivityentry(dayid,activityid);
String[] from = { "name", "calorie" };
int[] to = { android.R.id.text1, android.R.id.text2 };
final SimpleAdapter adapter = new SimpleAdapter(this,
list,android.R.layout.simple_list_item_2, from, to);
listview.setAdapter(adapter);
```

Figure 5.17 Listview code

```
listview.setOnItemClickListener(new AdapterView.OnItemClickListener() {

    public void onItemClick(final AdapterView adapterView, View view, final int
position, long id) {
        final String data = adapterView.getItemAtPosition(position).toString();
        String[] separated = data.split(",");
        String[] namesplit = separated[1].split("=");
        final String jadi = namesplit[1].substring(0, namesplit[1].length() - 1);
        AlertDialog.Builder adb=new AlertDialog.Builder(AddBreakfast.this);
        adb.setTitle("Delete?");
        adb.setMessage("Are you sure you want to delete " + position);
        final int positionToRemove = position;
        adb.setNegativeButton("Cancel", null);
        adb.setPositiveButton("Ok", new AlertDialog.OnClickListener() {
            public void onClick(DialogInterface dialog, int which) {
                db.deletefoodDiaryentry(jadi, activityid);
                adapter.notifyDataSetChanged();
            }
        });
        adb.show();
    }
});
}
```

Figure 5. 18 OnClickListener code

The list will show two data the name of the food and the value of calorie they have eaten. User can easily scroll to see and delete the data if already been inputted.

Figure 5.19 is used to delete the entry of food from the activity page. It used onclicklistener to listen when user click one of the entry of the list. Then it will take the data from the list and match it with the data in the database if match are found than the data will be deleted from the database.

```

public boolean onCreateOptionsMenu(Menu menu)
{
    MenuInflater menuInflater = getMenuInflater();
    menuInflater.inflate(R.menu.subactivitemenu, menu);
    prefs = getSharedPreferences("com.tesis.martinsitorus.dietproject20",
MODE_PRIVATE);
    editor = prefs.edit();
    return true;
}

public boolean onOptionsItemSelected(MenuItem item)
{
    switch (item.getItemId()) {
        case R.id.menu_add:
            Toast.makeText(AddBreakfast.this, "Add is Selected",
Toast.LENGTH_SHORT).show();
            prefs.edit().putInt("activitycode", 1).commit();
            Intent intent = new Intent(this, FoodSelection.class);
            startActivity(intent);
            return true;

        default:
            return super.onOptionsItemSelected(item);
    }
}
}

```

Figure 5. 19 Activity Action Bar Code

DietProject utilize Action bar in the program. Action bar are used to make uniformity in menu bar. In this menu user can add new input to the activity by selecting add food. User also can choose to add event that will make an entry to the calendar apps in google. Users also can add event to the calendar. If they wanted to

add an event to the calendar in order to make reminder about what to eat or what to workout.

5.3.5. Food List / Workout List

When User adding food / workout to their Activities they will be shown a list of food that already saved in the database. In order to make better performance FoodList class doesn't have preset list instead its popping new list layout to its correspondence layout mapping.

To show the user the food that have already been inputted to Diet Project's databases uses Listview. This view will show the user the list of food that have been inputted to Diet Project's databases. The list will load from table Food_Diary in the Database. In the activity page the listview are using the simple adapter with the scheme of simple_list_item_2.

Using this kind of approach the user can expect stability and more streamlined user experience.

The list will show two data the name of the food and the value of calorie they have eaten. User can easily scroll to see and delete the data if already been inputted. Too

```
final ListView listview = (ListView) findViewById(R.id.list_view);
ArrayList<Map<String, String>> list = db.getAllFoodSpecial();
String[] from = { "name", "calorie" };
int[] to = { android.R.id.text1, android.R.id.text2 };
SimpleAdapter adapter = new SimpleAdapter(this,list,android.R.layout.simple_list_item_2, from, to);
listview.setAdapter(adapter);
```

Figure 5.20 listview code

choose the desired food user can just tap the desired food they want. The Implementation Can be seen In Figure 5.20

User can added their desired food by simply tap the food they want using the onClickListener (Figure 5.21) functions.

```
listview.setOnItemClickListener(new AdapterView.OnItemClickListener() {
    public void onItemClick(AdapterView adapterView, View view, int position, long id) {
        String data = adapterView.getItemAtPosition(position).toString();
        String[] separated = data.split(",");
        String[] namesplit = separated[1].split("=");
        String jadi = namesplit[1].substring(0, namesplit[1].length() - 1);
        prefs.edit().putString("foochonam", jadi).commit();
        showEditDialog();
    }
});
}
```

Figure 5.21 onClickListener Code

When user tap the desired food then it will ask the student to input the portion of what they ate or workout.

```
nam = String.valueOf(namefood.getText().toString());
cal = Integer.parseInt(caloriefood.getText().toString());
port = Integer.parseInt(portionfood.getText().toString());
portnam = String.valueOf(portnamefood.getText().toString());

WorkoutModels fm = new WorkoutModels(nam, cal, port ,portnam);
food_id = db.createWorkout(fm);
```

Figure 5.22 Add new item code

5.3.6. Add new items

If the users cannot find the food/workout they looking for users can input their own.

Users must provide the amount of calorie and amount of portion. The implementation can be seen at Figure 5.22